

# Rosetta: Automating First-Principles Performance Modeling Using Multi-Agent LLMs

Karthikeyan Sankaralingam, NVIDIA

**Abstract**—Analytical performance models — derivations of throughput or speedup from hardware parameters — make claims independently verifiable and expose binding constraints, yet rarely accompany architecture papers because building one by hand takes weeks of expert effort. We present Rosetta, a multi-agent LLM pipeline that automatically generates first-principles analytical models from research paper PDFs. Given a paper as sole input, Rosetta produces a mathematical specification, an executable Python model, and a plain-English interpretation — all autonomously, with zero human intervention. The formalization process itself is the primary value: it surfaces implicit assumptions and identifies missing parameters. Four design decisions address failure modes of naïve LLM-based generation: a scientific constitution that prohibits circular reasoning, verify-repair loops with independent critic agents, dual verification separating functional correctness from scientific validity, and a best-of- $N$  ensemble that exploits LLM stochasticity.

We evaluate Rosetta across three complementary tracks: expert evaluation of 12 landmark papers (CS1), automated scoring of 97 unfiltered ISCA 2025 and HPCA 2026 papers (CS2), and author self-evaluation by six active research groups (CS3). Across CS1, specification quality scores 4–5/5 on 10 of 12 papers with zero significant hallucinations; across CS2, 56% of fit-screened papers reach Tier A insight quality. The strongest finding comes from CS3: Rosetta’s output led to revised claims and new experiments in active submissions, and five of six author-evaluators said they would use it again.

## I. INTRODUCTION

An analytical performance model — a closed-form or small-program derivation that predicts a system’s throughput, latency, energy, or speedup from hardware parameters and workload characteristics — is one of the most useful artifacts a computer architecture paper can provide. It makes transparent questions that cycle-level simulators can also address but require significant instrumentation and infrastructure to answer: *why* performance bounds exist, which parameters are the binding constraints, and how results would change under different conditions. Such models are invaluable across the research lifecycle. They serve readers by verifying claims independently, help authors debug designs alongside simulation, provide self-contained reproducible artifacts, allow industry practitioners to share architectural insights without exposing proprietary simulation infrastructure or IP (e.g., Anton [69], a molecular dynamics ASIC evaluated in our case-study), and serve as artifacts that guide follow-on work.

Despite this value, rigorous analytical models rarely accompany architecture papers. Building one by hand requires weeks of skilled effort, mathematical fluency, workload and hardware intuition. Consequently, the research community recognizes

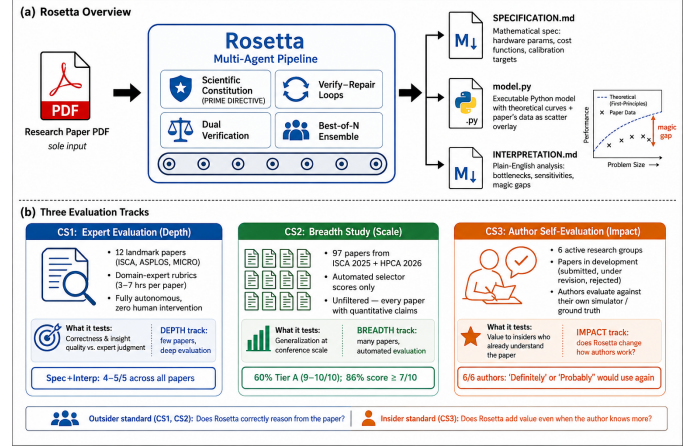


Fig. 1: Rosetta and Paper Overview

the value of analytical models but overwhelmingly relies on simulation or measurement results instead.

We present **Rosetta**, a closed-loop multi-agent pipeline that **automates** first-principles analytical model generation from research paper PDFs (Figure 1). Given a PDF as its sole input, Rosetta produces three artifacts: (i) `SPEC.md`, a mathematical specification of hardware parameters and cost functions; (ii) `model.py`, a self-contained executable Python model; and (iii) `INTERPRETATION.md`, a plain-English explanation of the model’s insights. A central concept is to quantify any discrepancy between what the paper’s described mechanism can achieve from first principles and its reported results, highlighting either unstated assumptions, omitted optimizations, or conservative claims. Just as code and data releases became standard practice, Rosetta-generated artifacts can make performance claims independently verifiable by any reader, without access to the authors’ simulation infrastructure.

We evaluate Rosetta in fully autonomous mode on published research papers—a deliberate stress test. A common misconception is that a camera-ready PDF is a perfectly precise and complete artifact, rendering automated modeling redundant. In reality, published papers are highly compressed and routinely elide implicit assumptions or obscure hardware parameters. By succeeding on these sparse documents, we establish a lower bound on Rosetta’s utility. If it can derive an independent, verifiable artifact from a compressed PDF to serve readers and reviewers, it is even more capable when applied to the ongoing, parameter-rich drafts of active researchers (as demonstrated in Case Study 3) or the highly detailed technical specifications used in industry. More broadly, the core question Rosetta answers for any technical document is: *did I build*

what I think I built, and are the benefits and limitations what I claimed they were? Automating this process raises four challenges: LLMs defaulting to circular reasoning (using reported results as inputs rather than calibrating against them), low-quality single-shot specifications, code that faithfully implements scientifically invalid specs, and LLM stochasticity. Rosetta addresses these via a **scientific constitution** (the PRIME DIRECTIVE) enforcing first-principles derivation, a **verify-repair loop** with independent critic agents, **dual verification** separating functional correctness from scientific validity, and a **best-of- $N$  ensemble** to ensure output quality and insight.

Recent work has begun to argue for GenAI-assisted discovery in architecture more broadly [24], [68]; Rosetta addresses one specific, thus far unresolved tool problem in that emerging space.

### Contributions

- **Rosetta**: an end-to-end multi-agent pipeline that produces first-principles analytical performance models from research paper PDFs, with a median end-to-end wall-clock time of 72 minutes, reducible to  $\sim 25$  minutes by running ensemble instances in parallel. It is released open-source with this paper, including all code, prompts, and evaluation artifacts, with Anthropic API implementation.
- **The PRIME DIRECTIVE**: a shared scientific constitution enforcing first-principles derivation, calibration-as-overlay, self-containment, and baseline vs. proposed comparison across all agents. We show empirically that removing the constitution degrades model quality substantially.
- **A verify-repair architecture with dual verification**: independently separating functional correctness (spec-to-code fidelity) from scientific validity (constitution compliance), closing a failure mode where a faithful implementation of a bad spec passes single-verifier review.
- **Evaluation across three case study (CS) tracks**: CS1 evaluates 12 landmark architecture papers against domain-expert rubrics; CS2 applies Rosetta in breadth to 97 ISCA 2025 and HPCA 2026 papers without expert evaluation; CS3 gives Rosetta to 6 researchers and collects author self-evaluations.
- **CAAM-Bench**: the first benchmark for evaluating analytical model generators on architecture papers — 12 landmark papers with human-vetted reference artifacts (SPECIFICATION.md and model.py) and a four-dimension scoring rubric. Rosetta’s CS1 scores serve as the published baseline for future tools.

### Key Findings

- **The formalization itself is the primary value.** Every CS3 author named SPECIFICATION.md — not the quantitative model — as the most valuable output. The act of reverse-engineering a paper’s performance structure into a mathematical specification surfaces implicit assumptions, flags missing parameters, and produces a readable analytical skeleton — value that is decoupled from whether the model achieves numerical accuracy. In two CS3 cases, Rosetta’s

formalization output directly caused changes to the paper: a simulation methodology comparison in one case, and a sharpened separation of GPU and custom-accelerator contributions in the other.

- **Human evaluators confirm correctness and insight.** Across 12 CS1 papers, independent domain-competent researchers scored specification quality at 4–5/5 on 10 of 12 papers with zero significant hallucinations, and rated 11 of 12 papers at 4/5 or 5/5 overall usefulness. No evaluator said “No” to future use.
- **Rosetta generalizes at scale.** In CS2, 60% of 97 unfiltered conference papers score in Tier A (9–10/10) on the automated selector’s insight quality metric, and 86% score 7/10 or above.
- **Researchers want to use it again — and it changed their papers.** Five of six CS3 evaluators said “Definitely” for future use; four had concrete paper impact (revised claims, new experiments, sharpened exposition). The adoption signal is not about numerical accuracy: as one evaluator put it, “No reviewer would have the time to write a simulation to validate a paper’s empirical results, but with Rosetta it’s a 5-minute task.” Another noted it “may have helped us earlier for idea development and eliminating less promising ideas.”

**Paper organization.** Section II provides Rosetta’s problem definition and design goals. Section III describes the system design, including empirical evidence for each design decision (Section III-G). Section IV frames the evaluation methodology. Sections V–VII present the three case study tracks. Section VIII surveys related work. Section IX concludes.

## II. THE PROBLEM FORMULATION

The standard artifact of architecture evaluation is a simulator or measurement harness — a tool that produces performance numbers but embeds its assumptions in code rather than exposing them mathematically. When a paper’s performance claims rest on such tools, independent verification requires access to the full infrastructure. Rosetta’s task is to produce the complementary artifact: a mathematical specification and executable model derived solely from the paper’s prose, against which the paper’s claims can be independently checked. We define this task as follows.

**Definition 1** (Analytical Model Generation). *Given a research paper PDF  $P$  describing a computer architecture or systems contribution with performance claims, produce: (i) a mathematical specification  $S$  defining the hardware parameters, workload model, and analytical cost functions for both the baseline and proposed systems, derived from the paper’s prose without using reported results as inputs; (ii) an executable model  $M$  implementing  $S$ ; and (iii) an interpretation  $\mathcal{I}$  reporting the model’s findings, including any quantified discrepancy between  $M$ ’s first-principles prediction and the paper’s reported results.*

The key constraint is *first-principles derivation*:  $M$  must compute performance from physical parameters ( $S$ ), not from

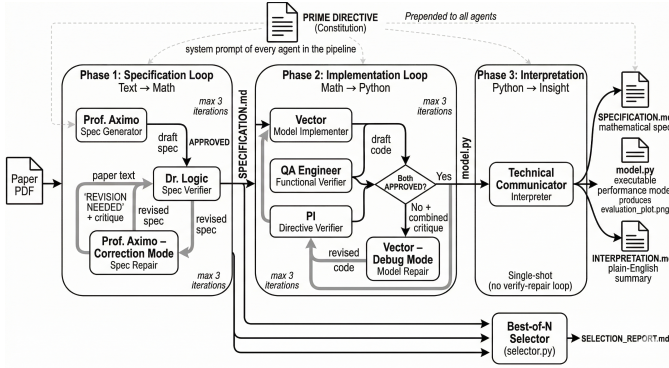


Fig. 2: Rosetta pipeline. Three sequential phases transform a paper PDF into a mathematical spec, executable model, and interpretation. Phases 1 and 2 run verify-repair loops of up to three iterations. The full pipeline runs  $N=3$  times; a selector agent picks the best run.

the paper’s reported results. The paper’s results appear in  $M$ ’s output as calibration scatter points — a visual representation of whether the first-principles prediction and the reported result agree. When they disagree significantly, the discrepancy is the *magic gap*. This formulation reflects Box’s principle that “all models are wrong, but some are useful” [8]: the goal is not numerical precision but mechanistic understanding that makes the paper’s performance claims independently auditable.

This formulation has a natural failure mode: if the paper does not expose the hardware parameters needed to derive performance from first principles,  $M$  cannot be fully derived. It can still produce bounds, calibration-mode estimates labeled as such, and a diagnostic listing of which parameters are missing. This is the correct behavior when the paper’s study reports measured speedups without exposing the underlying hardware configuration.

### III. SYSTEM DESIGN

Rosetta is a closed-loop pipeline that transforms a research paper PDF into three artifacts: a mathematical specification (SPECIFICATION.md), an executable Python performance model (model.py), and a plain-English interpretation (INTERPRETATION.md). Figure 2 shows the overall architecture. The pipeline is organized into three sequential phases, each driven by one or more large language model (LLM) agents operating under a shared scientific constitution. To mitigate LLM stochasticity, the entire pipeline runs multiple times and a selector agent chooses the best output (Section III-E). The complete system — including all agent prompts, the PRIME DIRECTIVE constitution, and evaluation artifacts — is released as open-source with this paper; the prompts are the primary design artifact and are integral to reproducibility. A concise usage guide accompanies the release to help users interpret outputs without reading this paper or becoming LLM experts.

#### A. Phase 1: Specification (Text → Math)

The goal of Phase 1 is to reverse-engineer a rigorous mathematical specification from the paper’s prose — the “physics” of the described system expressed as concrete variables, cost functions, and workload models.

**Generation (Prof. Aximo).** The specification generator, characterized as a rigorous expert in analytical performance modeling who “only trusts variables, equations, and concrete data points,” reads the full paper text and produces a structured document containing: (i) all hardware parameters with Python-compatible names and suggested value ranges (e.g., `dram_bandwidth_gbs = 51.2`); (ii) a workload model specifying the synthetic input distribution the model will use; (iii) analytical cost functions for both the *baseline* and *proposed* systems, derived bottom-up from hardware constraints; (iv) calibration targets — concrete data points extracted from the paper’s figures and tables, formatted as `(input_state, claimed_output, source)` tuples; and (v) an initial “magic gap” hypothesis noting any visible discrepancy between what the mechanism implies and what the paper reports.

**Verification (Dr. Logic).** An independent verifier — a “pedantic senior technical reviewer” — audits the specification against the original paper and the shared constitution (Section III-D). It checks for missing variables, hallucinated formulas, absent calibration data, and — most critically — circular reasoning: using the paper’s own reported results as formula inputs rather than as calibration overlays. The agent responds with exactly `APPROVED: <reason>` or `REVISION NEEDED: <detailed critique>`. The complete critique is saved to disk regardless of outcome, creating an audit trail.

**Repair loop and audit trail.** If the spec is rejected, the generator agent is re-invoked in “Correction Mode” with the original paper, the rejected spec, and Dr. Logic’s critique. The loop runs for up to three iterations; in practice, most specifications converge within one or two. Every intermediate artifact is saved (draft specs, draft models, all critique files), providing full transparency: a researcher can trace exactly how each conclusion was reached, and a diagnosis tool evaluates whether each iteration made genuine progress.

#### B. Phase 2: Implementation (Math → Python)

Phase 2 translates the approved specification into a self-contained, executable Python performance model.

**Implementation (Vector).** The implementer agent — a “brilliant Python data scientist and simulation engineer” — receives both the approved specification and the original paper text and produces a Python script with a fixed structure: (i) a `Config` dataclass holding all parameters from the spec; (ii) `cost_baseline()` and `cost_proposed()` functions implementing the spec’s cost formulas exactly; (iii) a simulation loop sweeping over the parameter ranges defined in the spec; and (iv) a `matplotlib` visualization with a dashed blue baseline curve, a solid green proposed curve, and red X scatter markers for the paper’s claimed results, saved as

evaluation\_plot.png (never `plt.show()`, since the pipeline runs headless). Generated code is syntax-validated via Python’s `ast.parse()` before saving.

**Dual verification.** Phase 2 uses *two independent verifiers* because code quality has two orthogonal failure modes. The **QA Engineer** performs a line-by-line functional audit: do variable names match the spec, are formulas implemented correctly, will numpy broadcasting work? The **PI (Principal Investigator)** performs a scientific validity audit: is there a baseline vs. proposed comparison, are calibration overlays present, is the code self-contained, and — most importantly — is performance derived from physical constraints rather than from table data? The PI’s test is concrete: “If you removed all Table/Figure data from the code, would it still calculate performance? If not, it is violating first principles.” Both agents must return `APPROVED` for the loop to exit; either rejection triggers a repair.

**Repair loop.** The implementer agent, in “Debug Mode,” receives both the QA Engineer’s and PI’s critiques simultaneously and rewrites the model from scratch addressing all issues. The loop again runs for up to three iterations, saving all draft versions and critique files as an audit trail.

### C. Phase 3: Interpretation (Python → Insight)

The final phase is a single-shot generation by a Technical Communicator agent. It reads the final `SPECIFICATION.md` and `model.py` and produces a plain-English document covering: the workload the model assumes; how the baseline is modeled and what it implies; the proposed mechanism and what changes; the critical variables and threshold conditions where the proposed system wins or loses; and any gaps between the theoretical curves and the paper’s claimed results. The tone guidance is “insightful and explanatory — explain model semantics, not Python syntax.” This phase has no verify-repair loop: interpretation quality improves less from iteration than specification or code correctness.

### D. The Scientific Constitution

Every agent in the pipeline receives a shared document — the **PRIME DIRECTIVE** — prepended to its role-specific prompt before any LLM call. This constitution defines four immutable laws that govern all agents:

- 1) **First-Principles Derivation.** Models must derive performance from physical and architectural constraints, never by scaling or fitting reported numbers. The distinction is illustrated with explicit code examples (Figure 3). The forbidden pattern takes a paper’s reported throughput and divides by the number of units to get per-unit throughput, then multiplies back — validating the paper’s claims against scaled versions of the same claims. The required pattern derives throughput from hardware parameters (core count, frequency, memory bandwidth), then *compares* the result to the paper’s reported values as an independent check.
- 2) **Calibration (The Truth Overlay).** Agents must extract specific data points from the paper’s figures, tables, and text

**Forbidden (circular):** `per_vcu = paper_reported_20vcu / 20; throughput_8vcu = per_vcu * 8`

**Required (first principles):**

```
mpix_per_core = pixels * fps / 1e6    (from Section 3.1)
compute_bound = mpix_per_core * cores_per_vcu
theoretical_max = min(compute_bound,
memory_bound)
efficiency = paper_claimed / theoretical_max
(15% gap!)
```

Fig. 3: The PRIME DIRECTIVE’s first-principles law illustrated. The forbidden pattern validates paper results against scaled versions of themselves — no independent derivation. The required pattern derives performance from hardware specs and then *compares* the result to the paper’s reported value.

and plot them as scatter markers on top of the theoretical curves. If a claimed data point sits above the theoretical curve, the paper is asserting performance that its described mechanism cannot support — this is the “magic gap.”

- 3) **Self-Containment.** The final `model.py` must run as `python model.py` with no arguments, no external files, and no network access. It generates its own synthetic workload data from distributions defined in the specification.
- 4) **Baseline vs. Proposed.** Every analysis must model both the status quo and the new contribution side-by-side. A proposed system analyzed in isolation — without a baseline — is scientifically meaningless.

The constitution is the mechanism by which Rosetta enforces scientific rigor across heterogeneous agents. Without it, agents routinely produce plausible-looking but methodologically invalid models — a finding confirmed by ablation.

### E. The Best-of-N Ensemble

LLM outputs are stochastic: the same pipeline on the same paper produces different specifications, models, and interpretations across runs. Some runs surface sharp insights; others miss key bottlenecks or make incorrect assumptions. Drawing on recent findings in LLM self-consistency and Best-of- $N$  reasoning [9], [15], Rosetta runs the full three-phase pipeline  $N=3$  times on each paper and uses a selector agent to choose the best run.

The selector executes `python model.py` for each run to confirm it runs without errors, then evaluates three criteria in priority order: **Correctness** (primary; a model that crashes or is trivially wrong is disqualified regardless of other merits), **Insight Quality** (primary differentiator; does the run identify concrete bottlenecks, quantify magic gaps, and go beyond restating the paper?), and **Completeness** (secondary; are all subsystems modeled, are calibration targets included?). The selector produces a per-run evaluation report and a final recommendation with explicit justification.

**Ensemble quality in practice.** Across our case study papers, the three ensemble runs diverged *meaningfully* — not merely as minor numerical variations, but in modeling philosophy

and interpretation quality. Consider two failure modes we observed. In one run on the Anton paper, the model predicted performance 5–6 $\times$  higher than the paper claimed, yet the interpretation document declared this a validation success — a failure of critical thinking that the selector identified and rejected. In another run on the CraterLake paper, a bug in the baseline formulation made the proposed-vs-baseline comparison meaningless, despite the individual cost functions being correct.

The selector’s primary differentiator — insight quality — consistently surfaces the run that treats unexplained gaps as *signals to flag* rather than problems to rationalize away. In the Darwin analysis, only one of three runs correctly noted that the headline “15,000 $\times$  speedup” applies only to ONT\_2D reads and drops to 1,244 $\times$  for ONT\_1D reads due to error-rate cascade — a nuance buried in the paper’s details.

Notably, there is no consistent bias toward which run number wins: across our papers, runs 1, 2, and 3 each won in roughly equal proportion. This confirms that the ensemble is not redundant — each run is a genuinely independent draw from the model’s hypothesis space. The ensemble also provides containment when the verify-repair loop does not fully converge: in one case study, a flaw in the F1+ baseline model survived three spec critique rounds and was faithfully propagated into the model by the functional verifier — which checks code-to-spec fidelity, not spec-to-physics correctness. The selector identified the affected run and chose an alternate with a sound baseline derivation.

#### F. Agent Inventory and Implementation

Table I summarizes the eight agents in the system. The pipeline is implemented as a fully automated Python program that orchestrates LLM API calls, parses structured outputs, manages file I/O, and executes generated code — no interactive chat interface is involved. Six of the eight agents come in generator/verifier or generator/repair pairs — a deliberate separation of concerns where no agent ever evaluates its own work. All agents run at temperature 0.5; the ensemble selects the best output across three full pipeline runs (Section III-E). Ensemble runs are executed sequentially rather than in parallel to simplify debugging and resource management; total runtime remains practical.

**Model choice.** All experiments were run with Claude Opus 4.5 (training data cutoff May 2025). In our experiments, Gemini-2.5, Gemini-3, and ChatGPT series models provided inferior results on our task. Frontier model comparison is orthogonal to our contributions and is an area for future work; we expect that other models will improve over time and that Rosetta’s architecture will be able to leverage those improvements as they arise.

Output token budgets follow a consistent two-tier design: generator and repair agents start at 16,384 tokens, while verifier agents start at 8,192 tokens, reflecting that verification requires shorter structured responses than generation. All agents share a hard cap of 64,000 tokens. If an agent terminates due to a context-length error, it retries up to three times with

TABLE I: Rosetta agent inventory. All agents receive the PRIME DIRECTIVE prepended to their role-specific prompt.

| Agent              | Phase | Role                |
|--------------------|-------|---------------------|
| Prof. Aximo        | 1     | Spec generator      |
| Dr. Logic          | 1     | Spec verifier       |
| Prof. Aximo (CM)   | 1     | Spec repair         |
| Vector             | 2     | Model implementer   |
| QA Engineer        | 2     | Functional verifier |
| PI                 | 2     | Directive verifier  |
| Vector (DM)        | 2     | Model repair        |
| Tech. Communicator | 3     | Interpretation      |

a 1.5 $\times$  budget multiplier per attempt (16,384  $\rightarrow$  24,576  $\rightarrow$  36,864  $\rightarrow$  55,296, capped at 64,000), accommodating long papers and complex derivations without permanently inflating the budget.

#### G. Why Naive Design Fails

Each design decision in the pipeline addresses a specific, empirically observed failure mode. The examples below are from Case Study 1. To prevent evaluation bias, the papers used during Rosetta’s initial development (Cambricon-SR [50] and F1 [65]) were strictly excluded from our case studies; instead, they served as the formative testbeds that revealed the failure modes described below and helped develop and refine Rosetta’s current design.

**Constitution and spec verification.** Without the PRIME DIRECTIVE, LLMs default to circular reasoning: an unguided CraterLake run back-calculated efficiency from the paper’s claimed 11.2 $\times$  speedup and predicted an impossible 465 $\times$ . Careful prompting alone is insufficient; the constitution provides concrete code-level anti-patterns that all agents enforce. Even with the constitution, spec-level errors can survive generation and propagate faithfully into code. A flawed CraterLake spec assigned CraterLake-specific hardware (the CRB unit) to the F1+ baseline, which does not have one; the resulting `model.py` passed code verification because it perfectly matched its buggy spec. A dedicated spec verification loop that reads against the original paper — before any code is written — is the only mechanism that catches this class of error.

**Dual verification and ensemble.** Functional correctness and scientific validity are orthogonal. In one Anton run, the code passed functional verification because it matched its spec, but the PI agent caught that the primary computation used a table of paper results — a first-principles violation baked into the spec itself. A single verifier checking both axes reliably deprioritizes one. Finally, LLM stochasticity makes single-run output unreliable: on NeuRex, Run 1 predicted a 3 $\times$  slowdown; Run 2 crashed yet hallucinated an interpretation; only Run 3 produced a sound derivation. On Darwin, only one of three runs correctly decomposed the 15,000 $\times$  headline speedup into per-workload components. The best-of- $N$  ensemble with a selector agent exploits this variance to reliably extract the most analytically sound output. Advanced single-agent environments (e.g., Claude Code) offer execution and

error-repair loops but collapse all of these safeguards: a single agent critiquing its own spec suffers from confirmation bias, cannot perform orthogonal dual verification, and precludes the stochastic variance the ensemble exploits. Multi-agent separation is not scaffolding; it is the core mechanism guarding against self-consistent but flawed reasoning.

#### IV. EVALUATION METHODOLOGY

Rosetta outputs mechanistic understanding, not numerical predictions alone. Evaluating it faces the same challenge as evaluating simulation frameworks [7], [40], [46] and benchmark suites [20]: no single automated metric captures whether an analytical artifact is correct, insightful, and useful. We evaluate across three complementary tracks — correctness, generalization, and practitioner adoption as detailed below across three types of case studies (CS).

CS1 (Section V) evaluates *correctness and insight*: 12 landmark papers spanning diverse domains, each assessed by an independent human evaluator using a structured rubric — analogous to validating a simulator against known microarchitectural behavior. CS2 (Section VI) tests *generalization*: Rosetta runs on 97 uncured ISCA 2025 and HPCA 2026 papers, scored by the automated selector, to confirm that CS1’s curated selection does not flatter the system. CS3 (Section VII) measures *practitioner value*: we give Rosetta to 6 active research groups whose authors have simulation ground truth and design intent absent from the PDF, and collect self-evaluations including concrete impact on the paper under development.

All results reported here are from Rosetta running in *fully autonomous mode* (a single launch of the multi-agent pipeline with only the paper PDF as input, zero human intervention, no post-hoc editing) to establish a *lower bound on capability*. In practice, the generated artifacts are self-contained and well-commented; researchers can load them into any AI-assisted coding environment (e.g., Claude Code, Cursor) alongside the PDF and iterate directly.

**Frontier model training data.** The CS1 landmark papers (published 2007–2023) are likely in Claude Opus 4.5’s training data (cutoff May 2025). However, Rosetta’s task — deriving a structured mathematical specification and executable model cognizant of hardware constraints is distinct from recall or summarization. CS2’s ISCA 2025 and HPCA 2026 papers were published at or after the training cutoff, and CS3’s papers are unpublished work no frontier model has seen. CS1 is thus the most conservative evaluation condition; CS2 and CS3 provide progressively stronger completely contamination-free evidence. The PRIME DIRECTIVE explicitly prohibits using reported results as formula inputs, so even memorized paper content cannot enter the derivation.

In the main text we present synthesized findings and representative deep dives. The full generated corpus will be released as a companion artifact.

#### V. CASE STUDY 1: CORRECTNESS AND INSIGHT

As described in Section IV, CS1 evaluates correctness and insight depth: *does Rosetta produce analytical models that*

*help an outsider understand a paper’s performance claims — correctly identifying bottlenecks, surfacing implicit assumptions, and quantifying gaps between the described mechanism and the reported results?* We curated a set of architecture papers spanning diverse problem domains, asked experienced architecture researchers to evaluate Rosetta’s outputs against both the paper and their own understanding using a structured rubric, and analyzed the results.

##### A. Experimental Setup

**Paper selection.** We selected 12 papers from ISCA, ASPLOS, and MICRO spanning seven domains (Table II), chosen to cover a wide difficulty spectrum: from papers with explicit analytical formulas (e.g., CraterLake) to those supported primarily by simulation (e.g., Warehouse-scale Video), with citation counts ranging from 39 to over 1,000.

**Evaluation protocol.** The automated selector chose the best of three ensemble runs per paper (Section III-E). Each human evaluator<sup>1</sup> then read or re-read the paper, examined all three output artifacts, and completed the rubric provided (3–7 hours per paper). Evaluators were architecture researchers with broad hardware-systems competence (self-rated 1–3/5 on domain-specific expertise), recruited independently of Rosetta, given no briefing on what constitutes “good” output, and instructed to be as critical as appropriate. They completed rubrics independently with no access to paper code, or simulators, and are not co-authors on this paper.

**Rubric structure.** This case study’s rubric covers four dimensions:

- **Specification quality** (Part 1): correctness of core relationship extraction, richness and completeness (1–5), hallucination check, and quality of the plain-English interpretation (1–5).
- **Model quality** (Part 2): execution, spec fidelity, baseline vs. proposed comparison, calibration overlay, self-containment, code clarity (1–5).
- **Insight value** (Part 3): parametric understanding (1–5), assumption surfacing (1–5), insight beyond reading the paper alone (1–5), and gap identification.
- **Overall assessment** (Part 4): overall usefulness (1–5), best concrete output, biggest failure, and whether the Evaluator would use Rosetta again on papers in this area.

Parts 1d, 3a–3c, and 4a require the most evaluator judgment and show the greatest variance across papers.

##### B. NeuRex (Neural Rendering) Deepdive

We begin with a look into one of the evaluator reports to illustrate the depth and nature of the insights produced by Rosetta, and then synthesize across all 12 papers to identify recurring patterns and conditions that predict when Rosetta adds the most value.

NeuRex [44] is a custom hardware accelerator for neural radiance field (NeRF) rendering (ISCA 2023) that achieves

<sup>1</sup>We use “human evaluator” throughout to distinguish the independent researchers who assessed Rosetta’s outputs from the automated selector and verifier agents within the pipeline.

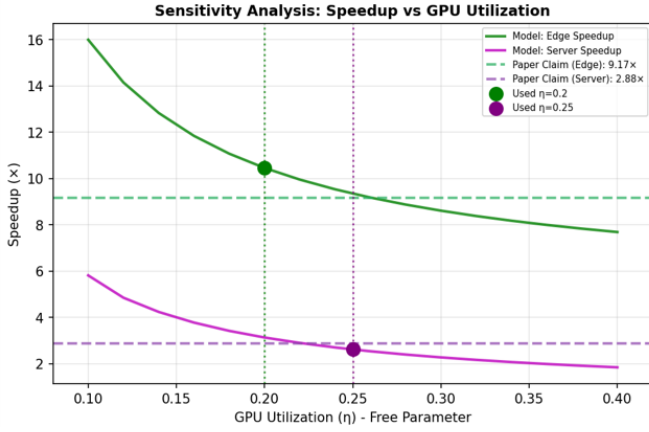


Fig. 4: Sensitivity analysis from the NeuRex model: predicted speedup vs. GPU utilization  $\eta$  for the Edge (Xavier NX, green) and Server (RTX 3070, purple) configurations. Dashed lines show the paper’s claimed speedups.

**From SPECIFICATION.md — Hardware Parameters (excerpt):**

| Variable                     | Edge   | Server | Source       |
|------------------------------|--------|--------|--------------|
| $N_{IGU}$ (Index Gen. Units) | 8      | 64     | \$4.4, Tab 4 |
| $S_{GC}$ (Grid cache)        | 64 KB  | 64 KB  | Tab 4, \$6.4 |
| $S_{SB}$ (Subgrid buffer)    | 128 KB | 128 KB | Tab 4        |
| $N_{SA}$ (Systolic arrays)   | 1      | 16     | \$5, Tab 4   |
| $f_{clk}$                    | 1 GHz  | 1 GHz  | \$5          |

**Baseline GPU parameters:**

|                     | Xavier NX | RTX 3070 | Source               |
|---------------------|-----------|----------|----------------------|
| $S_{L2}$ (L2 cache) | 256 KB    | 4 MB     | Sec 3.4, public spec |
| $BW_{mem}$          | 51.2 GB/s | 448 GB/s | Public spec          |

$BW_{Eff}(GPU): \eta_{BW,GPU} = B_{entry}/B_{cacheline} = 4/64 = 0.0625$   
Source: Sec 3.4, “each hash entry access only uses 4 out of 64 bytes.”

Fig. 5: Excerpt from the generated SPECIFICATION.md for NeuRex. Every parameter is traced to a paper section; the bandwidth efficiency is derived from hardware fundamentals.

9.17 $\times$  speedup over an edge GPU and 2.88 $\times$  over a server GPU using restricted hashing to shrink the multi-resolution hash table working set from 2 MB to 32 KB. It earned the highest scores across all CS1 dimensions (Spec, Model, Insight, Overall all 5/5). To illustrate exactly what Rosetta generates, we show concrete excerpts from the NeuRex artifacts before discussing the findings.

**What Rosetta generates: artifact excerpts.** Figure 5 shows a representative excerpt from the generated specification: the hardware parameter table for the NeuRex accelerator and baseline GPUs, with every value traced to a specific paper section. Figure 6 shows the corresponding model.py implementation of the GPU bandwidth-efficiency derivation — the core first-principles calculation that drives the model’s predictions. The full specification is 643 lines; the full model is 1,177 lines of self-contained Python. Figure 4 is one of the graphs produced by the model (details shortly). All three artifacts (spec, model, interpretation) are generated autonomously typically in 30 to 40 minutes.

**Key analytical finding.** Rosetta’s primary contribution for

**From model.py — GPU bandwidth efficiency (excerpt):**

```
def compute_gpu_bandwidth_efficiency(
    cacheline_bytes, bytes_per_entry):
    """From Sec 3.4: 'each hash entry access
    only uses four out of 64 bytes'"""
    return bytes_per_entry / cacheline_bytes
    # 4/64 = 0.0625

    # Cache fit analysis (first principles):
    # Xavier NX: 2 MB table > 256 KB L2 -> misses
    # RTX 3070: 2 MB table < 4 MB L2 -> fits
    table_fits = gpu.l2_cache >= hash.table_size
    # Speedup = T_gpu / T_neurex (NOT calibrated)
    # eta_util swept in [0.10, 0.40]
```

Fig. 6: Excerpt from the generated model.py for NeuRex. The code derives performance from hardware parameters; the paper’s reported results appear only as scatter overlays for comparison, never as formula inputs.

NeuRex is a first-principles derivation of why GPUs struggle with multi-resolution hash encoding:

$$\eta_{BW,GPU} = \frac{B_{entry}}{B_{cacheline}} = \frac{4 \text{ bytes}}{64 \text{ bytes}} = 6.25\%$$

Each hash table entry stores two 16-bit features, but GPU memory fetches 64-byte cache lines, wasting 60 of every 64 bytes. This structural inefficiency explains why NeuRex-Edge (9.17 $\times$ ) outpaces NeuRex-Server (2.88 $\times$ ) by such a wide margin: the Xavier NX’s 256 KB L2 cache cannot hold the 2 MB hash table, forcing every lookup to LPDDR4 main memory, while the RTX 3070’s 4 MB L2 already partially solves the problem NeuRex is designed to address. The model sweeps GPU utilization  $\eta \in [0.10, 0.40]$  without calibrating to the paper’s claims; both speedup figures fall within the predicted band (Figure 4). The human evaluator captured the mode of value precisely:

*“Makes plain from base principles how the new accelerator outpaces GPUs. It provides upfront analysis of the paper’s claims and exposes the gaps/withheld information of the paper.”*

### C. Aggregate Results

Table II summarizes evaluation scores across the 12 papers, which we analyze further below.

**Spec and interpretation quality is the most reliable output.**

Across all 12 evaluations, Spec+Interpretation quality scores range from 3 to 5, with ten of twelve papers at 4 or above. Human evaluators found that Rosetta reliably identifies the correct hardware parameters, constructs analytically reasonable cost functions, and produces an interpretation document that genuinely helps a newcomer orient to the paper. One human evaluator noted: “I found myself using the interpretation to help guide me through a re-read of the paper.” Zero expert evaluations reported significant hallucinations; four reported minor ones (CraterLake: assumed register file port counts and an arbitrary F1+ bandwidth value; Darwin: a bandwidth unit misinterpretation in the de novo throughput path; Warehouse: assumed combined read/write accounting;

TABLE II: CS1 scores (12 papers, 1–5 scale). **S**=Spec+Interp, **M**=Model, **R**=Richness, **P**=Parametric, **A**=Assumption surfacing, **I**=Insight>Paper, **O**=Overall. **?**=Use Again (**D**=Definitely, **P**=Probably, **U**=Unsure).

| Paper           | Domain     | S | M | R | P | A | I | O | ?        |
|-----------------|------------|---|---|---|---|---|---|---|----------|
| BASALISC [22]   | FHE        | 5 | 5 | 4 | 4 | 4 | 4 | 5 | <b>D</b> |
| Darwin [74]     | Genomics   | 4 | 5 | 5 | 3 | 2 | 5 | 5 | <b>D</b> |
| CraterLake [66] | FHE        | 4 | 3 | 4 | 4 | 1 | 4 | 5 | <b>P</b> |
| Warehouse [61]  | Video      | 4 | 4 | 3 | 5 | 3 | 3 | 4 | <b>U</b> |
| Anton [69]      | Mol.Dyn.   | 4 | 5 | 4 | 2 | 3 | 4 | 4 | <b>P</b> |
| GenAx [21]      | Genomics   | 4 | 4 | 4 | 4 | 3 | 4 | 5 | <b>P</b> |
| Genesis [26]    | Genomics   | 5 | 4 | 4 | 5 | 3 | 4 | 5 | <b>P</b> |
| SeGraM [10]     | Genomics   | 3 | 2 | 3 | 3 | 1 | 3 | 4 | <b>P</b> |
| GZKP [54]       | ZK Proofs  | 3 | 5 | 4 | 3 | 3 | 3 | 4 | <b>P</b> |
| PipeZK [88]     | ZK Proofs  | 4 | 3 | 3 | 4 | 4 | 4 | 5 | <b>P</b> |
| NeuRex [44]     | Neur.Rend. | 5 | 5 | 4 | 4 | 4 | 5 | 5 | <b>D</b> |
| MD-Pipe [37]    | Mol.Dyn.   | 4 | 4 | 4 | 3 | 3 | 4 | 5 | <b>P</b> |

SeGraM: an HBM channel–accelerator topology assumption) that human evaluators considered inconsequential to the core derivation.

**Key takeaway:** *The specification and interpretation are Rosetta’s most reliable outputs, scoring 4+ on 10 of 12 papers with zero significant hallucinations.*

#### Model quality is strong but bounded by spec quality.

Model quality scores range from 2 to 5, with five papers at 5/5 (BASALISC, Darwin, Anton, GZKP, NeureX). In one confirmed case, the model corrects a minor spec-level error: Darwin’s `model.py` fixes an incorrect de novo throughput formula that propagated from the specification. In the opposite direction, Anton’s incorrect  $T_{\text{long-range}}$  formula is faithfully reproduced from the spec into the model, illustrating that the pipeline’s verify step catches scientific invalidity but does not independently audit arithmetic against the paper. The 3/5 scores for CraterLake and PipeZK both reflect spec-level gaps propagated into code: for CraterLake, the F1+ baseline incorrectly used CraterLake’s CRB unit cycles when F1+ has no such hardware; the single 2/5 (SeGraM) reflects an inadequate baseline model: the BitAlign accelerator derivation correctly follows Algorithm 1 line-by-line, and the 100× end-to-end latency gap is correctly identified as the central analytical finding, but the CPU comparison system is modeled as a generic processor rather than one implementing the MinSeed and BitAlign algorithms that SeGraM accelerates — a gap the paper’s own sparse baseline description makes difficult to close from the PDF alone; and flagged in the SPEC critique, and observed in INTERPRETATION. This confirms a key design constraint: model quality is bounded below by spec quality — a sound model cannot be built on an insufficient specification.

**Key takeaway:** *Model quality (2–5) is never higher than spec quality; the verify-repair loop’s primary leverage point is the specification phase.*

**Insight value is the most discriminating dimension.** The “insight beyond paper” score varies most widely (3–5) and is the strongest predictor of overall usefulness. Rosetta adds most value when: (i) the paper presents sparse mathematical detail but rich hardware description, forcing assumptions into the

open; (ii) the paper reports performance at a single operating point, and the parametric model reveals behavior across a design space. The CraterLake  $L$ -dependence sweep is the clearest example of (ii): the paper reports speedup at fixed benchmarks, but Rosetta produced a continuous speedup vs.  $L$  curve showing 4× at low  $L$  (shallow FHE) to 51× at high  $L$  (deep FHE) — a result not available from the paper’s tables alone. NeureX and Darwin are the two papers reaching 5/5 insight; in both cases the paper describes dense hardware mechanisms whose parametric interaction the model resolves into a navigable design-space map.

**Key takeaway:** *Insight value is highest when papers have rich hardware descriptions but sparse explicit math, and when parametric sweeps reveal design-space behavior hidden in point results.*

#### Where insight is bounded.

Three papers score 3/5 on insight, each for a distinct structural reason. Warehouse-scale Video illustrates a *parameter-withholding ceiling*: key configuration values are withheld for proprietary reasons, preventing Rosetta from explaining the 6.5× gap between theoretical peak and observed throughput. The Evaluator credited the model for correctly reaching this conclusion and flagging it: “the model knows why the model is lean — I don’t think a human could do any better.” GZKP illustrates a *component-divergence ceiling*: NTT results are confirmed analytically, but the MSM component model diverges substantially from empirical results, “calling more questions about itself than the paper it is meant to reflect.” SeGraM illustrates a *missing-baseline ceiling*: without a model for the CPU baseline that implements MinSeed and BitAlign, the comparative insight is limited to confirming the accelerator’s own scalability properties rather than explaining the relative speedup. Across all three, Rosetta correctly diagnoses the ceiling rather than producing spuriously confident output.

**Key takeaway:** *When insight is bounded, Rosetta identifies why — parameter withholding, component divergence, or missing baselines — rather than spuriously confident output.*

#### When Does Rosetta Add Most Value?

Across all 12 evaluations, Rosetta’s insight value is a function of the *paper’s mathematical disclosure posture*, not of Rosetta’s intrinsic capability. Crucially, the three papers scoring 3/5 on insight — Warehouse-scale Video, SeGraM, and GZKP — share a common characteristic: Rosetta correctly diagnosed its own ceiling rather than producing spuriously confident output, a property that makes its confirmed findings credible. Table III catalogs the eight recurring finding types observed across the 12 papers; four conditions consistently predict high insight value: (1) rich hardware description with sparse explicit math (forces assumptions into the open); (2) single-point results whose latent parametric structure the model reveals; (3) a gap between first-principles prediction and claimed results that characterizes rather than condemns; and (4) formalization as standalone value, where the specification delivers insight even when the quantitative model is imperfect.

TABLE III: Recurring finding types across CS1, with representative papers and concrete outputs.

| Finding type                                                    | Papers                     | What Rosetta produced                                                                                                                                                                   |
|-----------------------------------------------------------------|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Structural hardware bottleneck from first principles            | NeuRex, Genesis            | 6.25% BW efficiency (4/64 bytes/cache-line); $2.08\times$ achieved is within 1.4% of Amdahl ceiling                                                                                     |
| Parametric sweep reveals hidden design-space behavior           | CraterLake, NeuRex, PipeZK | $L$ -dependence curve ( $4\times \rightarrow 51\times$ ); $\eta$ -sweep bracketing claimed speedup without calibration; degradation from $197\times$ to $30\times$ across problem sizes |
| Module-to-system gap closed analytically                        | PipeZK, Genesis            | Amdahl prediction $5.2\times$ vs. paper’s $5.8\times$ ; ceiling identifies true performance floor                                                                                       |
| Formalization as standalone value (model imperfect)             | PipeZK                     | 5/5 Overall despite 3/5 Model; qualitative paper statements converted to quantified boundaries                                                                                          |
| Component-level audit: confirmation and divergence              | GZKP                       | NTT $O(N \log N)$ scaling verified; MSM sign-flip inverts speedup prediction                                                                                                            |
| Model corrects a spec-level paper error                         | Darwin                     | Incorrect de novo throughput formula caught in implementation phase                                                                                                                     |
| Paper withholds quantitative data; model makes claims checkable | NeuRex, Warehouse          | $\eta$ -sweep without calibration; proprietary ceiling correctly diagnosed                                                                                                              |
| Missing-baseline ceiling correctly diagnosed                    | SeGraM                     | $100\times$ latency gap attributed to undocumented optimizations; generic-CPU baseline limits comparative insight                                                                       |

#### D. CAAM-Bench: CompArch Analytical Model Benchmark

No standard benchmark exists for evaluating tools that automatically generate analytical models from architecture papers, making reproducible comparison across approaches or over time impossible. We release the CS1 evaluation set as **CAAM-Bench** (CompArch Analytical Model Benchmark) to fill this gap. CAAM-Bench has three components:

- **12 landmark architecture papers** spanning seven domains (Table II), selected to cover a wide difficulty spectrum from papers with explicit analytical formulas to simulation-heavy works, with citation counts from 39 to over 1,000.
- **Reference artifacts:** for each paper, a human-vetted SPECIFICATION.md and model.py produced by Rosetta and confirmed by an independent domain-expert evaluator. These serve as a quality baseline — a concrete target for future tools to match or exceed.
- **A scoring protocol:** the four-dimension CS1 rubric (specification quality, model quality, insight value, overall usefulness) applied to the spec and model produced by any tool or LLM on the same 12 papers. The rubric can be applied by human evaluators or, following CS2’s approach, by an automated scorer calibrated against the human-vetted reference. Rosetta’s scores (Table II) serve as the published baseline.

CAAM-Bench enables reproducible evaluation of future analytical model generators by providing a fixed paper corpus, reference artifacts, and a common scoring protocol. The 12 papers span sufficient domain diversity — FHE, genomics, molecular dynamics, ZK proofs, neural rendering, and video — that performance on any single domain is unlikely to generalize without genuine analytical depth. All benchmark materials are released with this paper.

#### VI. CASE STUDY 2: BREADTH STUDY

To confirm that CS1’s curated selection does not flatter the system, we ran Rosetta on 97 unfiltered papers from

ISCA 2025 (80) and HPCA 2026 (17). An independent fit judge excluded 17 papers unsuitable for first-principles analysis (scheduling heuristics, purely algorithmic work); the remaining **78 papers** form the reported corpus. Across these, mean Insight Quality is 8.2/10 and mean Correctness is 7.8/10; 56% reach Tier A (9–10) and 91% score 7+. We classified outcomes into three categories: **GAP-ONLY** (60, 77%) where the model quantifies a discrepancy without attributing a cause it cannot derive; **SHOWS-NEW-INSIGHT** (12, 15%) where the derivation surfaces a tension with a specific claim not visible from reading alone; and **VALIDATES** (5, 6%) where the derivation independently confirms the paper’s result. The median pipeline runtime is 73 minutes at  $\sim \$39$ /paper (\$3,800 total).

**Key takeaway:** Rosetta generalizes at conference scale, with 15% of papers surfacing analytical tensions not visible from reading the paper alone.

#### VII. CASE STUDY 3: RESEARCHER USE

As described in Section IV, CS3 tests practitioner value: *does Rosetta add value during active research?* We gave Rosetta to six research groups with papers at premier architecture venues (submitted, under revision, or accepted) and collected author self-evaluations using a structured rubric (Figure 7). Authors evaluated Rosetta’s output against their own simulation ground truth and design intent — context absent from the PDF and unavailable to CS1’s external evaluators. Titles and venues are redacted as the papers are active submissions<sup>2</sup>.

##### A. Researchers Want to Use Rosetta Again

We do not bury the lede. Five of six CS3 evaluators — including those who gave the lowest overall scores — said “Definitely” when asked whether they would run Rosetta on their next paper; the sixth said “Unsure” only because their

<sup>2</sup>All papers target top-3 architecture or systems venues.

paper was a simulation study at the hardest analytical ceiling. The impact these researchers describe goes beyond anything a single accuracy number can capture. SYMI’s evaluator: “No reviewer would have the time to write a simulation to validate a paper’s empirical results, but with Rosetta it’s a 5-minute task.” SgBend’s evaluator: “If we had used this from the beginning, we might have been able to have a more profound influence on the final work — it also may have helped us earlier for idea development and eliminating less promising ideas.” D-Com’s evaluator: “Made assumptions explicit, and the generated spec is useful.” The Mamba evaluator gave the only 5/5 spec richness in the CS3 set: “Did a great job comprehending the Einsums, the fusion taxonomy, and the 2D/1D reconfigurable array. Surprisingly well.” Even the GT evaluator, who rated overall usefulness at 2/5, confirmed that Rosetta’s bandwidth under-utilization derivation “is very similar to our simulation results.”

**Key takeaway:** Across six independent evaluations spanning five different system domains, the consistent signal is adoption intent — not because the models are perfect, but because the formalization process itself changes how researchers engage with their own work.

### B. Other Per-Paper Technical Findings

We summarize the key Rosetta-driven finding from each paper below.

**SYMI (MoE training):** Rosetta reproduced the paper’s communication-cost formulas and independently constructed a Zipfian-distribution training simulation — an approach the author had started and abandoned for the same reason (too many assumptions about the skewness parameter). The convergence ratio matched precisely (29% faster). The author noted that an earlier interactive run “could have been incorporated directly” into the paper.

**D-Com (LLM inference):** SPECIFICATION.md revealed that the paper’s exposition insufficiently separated the motivational GPU analysis from the proposed custom accelerator; the paper was updated to sharpen this boundary.

**Mamba (SSM accelerator):** Earned 5/5 spec richness and surfaced a terminological tension — whether “fully fused” is accurate when DRAM traffic exists at skip-connection boundaries.

**TOPO (sparsity/RT core):** Hardware pipeline captured correctly; workload abstractions too generic to match simulation, establishing the dataset-dependence boundary condition.

**SgBend (SpGEMM):** Strongest concrete impact: Rosetta identified an IPM lookup latency the authors’ own simulator had underestimated, directly motivating a new experiment added to the paper.

### C. A Graph Traversal Accelerator(GT)

We now highlight the graph traversal simulation study (the hardest analytical case) *This paper studies a graph traversal accelerator. The lead author compared Rosetta’s output against cycle-accurate simulation ground truth not present in*

*the PDF. The author rated the output 2/5; we report that as ground truth.*

**Reading synthesis value.** Before the performance model runs, Rosetta’s reading assistant independently flagged four analytical observations: the headline speedup over a prior FPGA baseline includes a clock-frequency component alongside the architectural improvement; benchmark selection was bounded by a wall-clock simulation budget, omitting larger instances; most selected instances fit on-chip, concentrating the off-chip scalability evidence in a single reduced-cache experiment; and the heuristic-solver comparison involves asymmetric capability sets. The author confirmed post-hoc that the paper was subsequently rewritten to address exactly these concerns — the reading synthesis had identified the same objections that program-committee reviewers later raised in writing.

### Quantitative model: one confirmed finding, one error.

The model derives baseline cycles per traversal step as  $t_{step}^{base} \approx 167$  cycles, implying memory bandwidth utilization of only  $\approx 3.8\%$  — the system is severely latency-bound, not bandwidth-bound. The author explicitly confirmed this against simulation. A second finding — that the accelerator has worse vertex-data cache hit rates than the baseline (60% vs. 70%) but better adjacency-list hit rates (85% vs. 65%) — was also verified against simulation and found to be wrong. We report this as a genuine model error. First-principles derivation closes to  $9.19\times$  speedup against the paper’s  $22.77\times$ , a  $2.48\times$  unexplained gap.

**Author verdict and scope implication.** The author rated overall usefulness at 2/5 and “Unsure” for future use, and offered the most precise diagnosis of the gap: not missing hardware parameters, but the LLM’s lack of algorithm-specific understanding of how the core propagation algorithm interacts with memory access patterns. This is a hard scope boundary: even a complete parameter checklist cannot bridge it. The author’s most actionable suggestion: “Adding a Q&A part with a planning phase could help make sure the model stays on track” — a pre-flight audit that enumerates which analytical inputs are and are not recoverable from the paper alone.

### D. Aggregate Results

Spec richness averages 4.0/5 with five of six at 4+, consistent with CS1’s finding that the specification is the pipeline’s most reliable phase. Every CS3 evaluator named SPECIFICATION.md as the most valuable output, independent of model accuracy. Four of six evaluations produced a documented change to the paper.

**Key takeaway:** The formalization pass adds value regardless of whether model.py closes numerically; four of six papers were changed as a direct result.

**The SPECIFICATION.md artifact is the most consistently valued output.** Every CS3 evaluator named it as the most valuable output, independent of model accuracy. SYMI’s spec enabled an independent rediscovery of the author’s own abandoned simulation; D-Com’s surfaced a structural exposition problem in the paper; SgBend’s introduced a new analytical technique for capturing data-dependent hardware behavior that

| Dimension                        | What it asks                                                                        |
|----------------------------------|-------------------------------------------------------------------------------------|
| <b>Core capture</b> (2a)         | Did Rosetta identify the key analytical relationships the author considers central? |
| <b>Spec richness</b> (2b)        | Does the spec capture baseline + proposed, parameters, operating conditions?        |
| <b>Interp quality</b> (2e)       | Does INTERPRETATION.md accurately explain the work to an outsider?                  |
| <b>Assumption surfacing</b> (3a) | Did formalizing expose implicit assumptions the author had not previously stated?   |
| <b>Parametric clarity</b> (3b)   | Did the model illuminate parameter sensitivity the author hadn't fully explored?    |
| <b>Impact on paper</b> (3c)      | Did Rosetta's output change any claim, section, or experiment?                      |
| <b>Overall usefulness</b> (5a)   | Holistic author rating                                                              |
| <b>Use again?</b> (4c)           | Would you run Rosetta during development of your next paper?                        |

| Paper                      | Domain     | (2a)   | (2b) | (2e) | (5a) | (4c)       |
|----------------------------|------------|--------|------|------|------|------------|
| <i>GT</i> <sup>*</sup>     | GraphTrav. | Partly | 4    | 3    | 2    | Unsure     |
| <i>SYMI</i> <sup>†</sup>   | MoE Sys.   | Mostly | 4    | –    | 3    | Definitely |
| <i>D-Com</i> <sup>‡</sup>  | LLM Inf.   | Yes    | 4    | 3    | 4    | Definitely |
| <i>Mamba</i> <sup>*</sup>  | SSM        | Yes    | 5    | 3    | 3    | Definitely |
| <i>TOPO</i> <sup>‡</sup>   | Sparsity   | Partly | 3    | 3    | 3    | Definitely |
| <i>SgBend</i> <sup>†</sup> | SpGEMM     | Partly | 4    | 5    | 4    | Definitely |

Fig. 7: CS3 rubric dimensions (top) and scores (bottom). Scored dimensions use 1–5; categorical: Yes/Mostly/Partly/No (2a) and Definitely/Probably/Unsure/No (4c). <sup>\*</sup>Rejected, being revised; <sup>†</sup>Accepted; <sup>‡</sup>New submission.

the evaluator intends to adopt. The formalization pass adds value regardless of whether `model.py` closes numerically.

**Key takeaway:** Every CS3 evaluator named *SPECIFICATION.md* as the most valuable output, independent of model accuracy.

## VIII. RELATED WORK

Rosetta sits at the intersection of analytical modeling, LLM-based scientific reasoning, and multi-agent system design. We survey each area, emphasizing how prior work addresses subsets of the problem Rosetta solves end-to-end.

**Analytical Modeling and Simulation** Analytical models like Roofline [17], [58], [81] and Amdahl’s Law [1], [25], [27], [29], [35] provide fundamental performance bounds. Deep learning models [14], [16], [34], [38], [56] apply IO-complexity and empirical scaling laws. More detailed first-principles processor [19], [39] and GPU models [5], [31], [32], [41], [43], [57], [71], [77], [87] rely on manual interval and reuse-distance analysis. Others tailor models to specific workloads or hybridize with simulation [2], [23], [28], [75]. However, all are manually constructed by experts. In contrast, cycle-level simulators [3], [7], [11], [53], [62], [67] require precise hardware specifications and binaries. Rosetta occupies a unique space: it *automates* the creation of analytical models directly from unstructured paper prose.

**LLM-Based Science and Peer Review** LLMs increasingly assist in scientific document understanding, including QA,

claim verification, and literature synthesis [4], [51], [72], [76], [79], [82]. Beyond comprehension, systems like The AI Scientist [52], FunSearch [63], AlphaGeometry [73], and formal provers [60], [84] attempt automated mathematical discovery and experimentation. LLMs are also explored for automated peer review [47], [48]. Rosetta fundamentally differs: it does not generate novel hypotheses, train models, or recommend paper acceptance. Instead, it operates as an analytical auditor, extracting parameters from existing text to mathematically verify claimed mechanisms against first principles.

**Multi-Agent Architectures and Verification** Rosetta leverages recent advances in LLM reasoning (Chain/Tree of Thought [80], [85] to mitigate hallucination [36]), code generation [13], [64], [78], [86], and multi-agent roles [30], [45], [83]. To ensure robustness, we build upon iterative self-refinement [33], [42], [55], [59], [70], multi-agent debate/evaluation [12], [18], [89], and Best-of- $N$  process verification [9], [15], [49]. However, while Constitutional AI [6] enforces behavioral norms via RLHF, Rosetta pioneers a *scientific constitution* (the PRIME DIRECTIVE) enforced via strict generator-verifier separation. Crucially, we introduce an orthogonal dual-verification step (functional vs. scientific validity) to prevent the specific failure mode where an LLM faithfully implements a circular, non-first-principles mathematical specification. While recent systems like PaperBanana [90] orchestrate specialized agents to automate the generation and critique of academic visual illustrations, Rosetta applies a similar multi-agent decomposition to the extraction of mathematical artifacts in architecture and systems.

## IX. CONCLUSION

Analytical performance models explain *why* performance bounds exist, yet they rarely accompany architecture papers. Rosetta automates this: given a PDF, it produces a mathematical specification, executable Python model, and plain-English interpretation with zero human intervention, using a scientific constitution, dual verification, and a best-of- $N$  ensemble to address the failure modes of naive LLM-based generation. Across three tracks, Rosetta demonstrates correctness (10 of 12 CS1 papers at 4–5/5 specification quality, zero significant hallucinations), generalization (56% Tier A across 78 unfiltered conference papers), and practitioner value (five of six CS3 researchers said “Definitely” for future use). The consistent finding is that formalization itself is the primary value: it surfaces implicit assumptions, flags missing parameters, and changes how researchers engage with their own work.

We release Rosetta open-source together with **CAAM-Bench**, the first benchmark for evaluating analytical model generators on architecture papers, comprising 12 human-vetted paper–artifact pairs and a four-dimension rubric to enable reproducible comparison of future tools. More broadly, Rosetta points toward a new community norm: architecture papers accompanied by a generated spec and model, making every performance claim independently auditable by any reader, without access to the authors’ simulation infrastructure.

## REFERENCES

- [1] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," *AFIPS Conference Proceedings*, vol. 30, pp. 483–485, 1967.
- [2] Y. Arafa, A.-H. Badawy, A. ElWazir, A. Barai, A. Eker, G. Chennupati, N. Santhi, and S. Eidenbenz, "Hybrid, scalable, trace-driven performance modeling of GPGPUs," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, 2021, pp. 1–15.
- [3] T. Austin, E. Larson, and D. Ernst, "Simplescalar: an infrastructure for computer system modeling," *Computer*, vol. 35, no. 2, pp. 59–67, 2002.
- [4] J. Baek, S. K. Jauhar, S. Cucerzan, and S. J. Hwang, "ResearchAgent: Iterative research idea generation over scientific literature with large language models," in *arXiv preprint arXiv:2404.07738*, 2024.
- [5] S. S. Baghsorkhi, M. Delahaye, S. J. Patel, W. D. Gropp, and W.-m. W. Hwu, "An adaptive performance modeling tool for GPU architectures," in *Proceedings of the 15th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*. ACM, 2010, pp. 105–114.
- [6] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon *et al.*, "Constitutional AI: Harmlessness from AI feedback," *arXiv preprint arXiv:2212.08073*, 2022.
- [7] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti *et al.*, "The gem5 simulator," *ACM SIGARCH Computer Architecture News*, vol. 39, no. 2, pp. 1–7, 2011.
- [8] G. E. P. Box, "Science and statistics," *Journal of the American Statistical Association*, vol. 71, no. 356, pp. 791–799, 1976.
- [9] B. Brown, J. Juravsky, R. Ehrlich, R. Clark, Q. V. Le, C. Ré, and A. Mirhoseini, "Large language monkeys: Scaling inference compute with repeated sampling," 2024. [Online]. Available: <https://arxiv.org/abs/2407.21787>
- [10] D. S. Cali, K. Kanellopoulos, J. Lindegger, Z. Bingöl, G. S. Kalsi, Z. Zuo, C. Firtina, M. B. Cavlak, J. Kim, N. M. Ghiasi, G. Singh, J. Gómez-Luna, N. A. Alser, M. Alser, S. Subramoney, C. Alkan, S. Ghose, and O. Mutlu, "Segram: a universal hardware accelerator for genomic sequence-to-graph and sequence-to-sequence mapping," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ser. ISCA '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 638–655. [Online]. Available: <https://doi.org/10.1145/3470496.3527436>
- [11] T. E. Carlson, W. Heirman, and L. Eeckhout, "Sniper: Exploring the level of abstraction for scalable and accurate parallel multi-core simulation," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2011, pp. 1–12.
- [12] C.-M. Chan, W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu, and Z. Liu, "ChatEval: Towards better LLM-based evaluators through multi-agent debate," *arXiv preprint arXiv:2308.07201*, 2023.
- [13] M. Chen, J. Twarek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," in *arXiv preprint arXiv:2107.03374*, 2021.
- [14] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann *et al.*, "PaLM: Scaling language modeling with pathways," in *Journal of Machine Learning Research*, vol. 24, no. 240, 2023, pp. 1–113.
- [15] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Twarek, J. Hilton, R. Nakano *et al.*, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.
- [16] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proceedings of the 36th Conference on Neural Information Processing Systems (NeurIPS)*, 2022, pp. 16 344–16 359.
- [17] N. Ding, M. Awan, and S. Williams, "Instruction roofline: An insightful visual performance model for gpus." Lawrence Berkeley National Laboratory (LBNL), Berkeley, CA (United States), 01 2021. [Online]. Available: <https://www.osti.gov/biblio/1844927>
- [18] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- [19] S. Eyerman, L. Eeckhout, T. Karkhanis, and J. E. Smith, "A mechanistic performance model for superscalar out-of-order processors," *ACM Transactions on Computer Systems*, vol. 27, no. 2, pp. 1–37, 2009.
- [20] M. Ferdman, A. Adileh, O. Koçberber, S. Volos, M. Alisafae, D. Jevdjic, C. Kaynak, A. D. Popescu, A. Ailamaki, and B. Falsafi, "Clearing the clouds: A study of emerging scale-out workloads on modern hardware," in *Proceedings of the 17th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2012, pp. 37–48.
- [21] D. Fujiki, A. Subramaniyan, T. Zhang, Y. Zeng, R. Das, D. Blaauw, and S. Narayanasamy, "Genax: A genome sequencing accelerator," in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, 2018, pp. 69–82.
- [22] R. Geelen, M. V. Beirendonck, H. V. L. Pereira, B. Huffman, T. McAuley, B. Selfridge, D. Wagner, G. Dimou, I. Verbaauwhede, F. Vercauteren, and D. W. Archer, "BASALISC: Programmable hardware accelerator for BGV fully homomorphic encryption," *Cryptology ePrint Archive*, Paper 2022/657, 2022. [Online]. Available: <https://eprint.iacr.org/2022/657>
- [23] X. Gong, C. Hu, and C.-C. Lim, "PAQSIM: Fast performance model for graphics workload on mobile GPUs," in *Proceedings of the 21st ACM SIGPLAN/SIGBED Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES)*. ACM, 2020, pp. 127–138.
- [24] R. Gupta, A. Jain, A. Gonzalez, A. Novikov, P.-S. Huang, M. Balog, M. Eisenberger, S. Shirobokov, N. Vü, M. Dixon, B. Nikolic, P. Ranganathan, and S. Karandikar, "Archagent: Agentic ai-driven computer architecture discovery," 2026. [Online]. Available: <https://arxiv.org/abs/2602.22425>
- [25] J. L. Gustafson, "Reevaluating Amdahl's law," *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.
- [26] T. J. Ham, D. Bruns-Smith, B. Sweeney, Y. Lee, S. H. Seo, U. G. Song, Y. H. Oh, K. Asanovic, J. W. Lee, and L. W. Wills, "Genesis: A hardware acceleration framework for genomic data analysis," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 254–267.
- [27] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2019.
- [28] S. Heo, S. Cho, Y. Kim, and H. Kim, "Real-time object detection system with multi-path neural networks," in *Proceedings of the 26th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2020, pp. 174–187.
- [29] M. D. Hill and M. R. Marty, "Amdahl's law in the multicore era," *Computer*, vol. 41, no. 7, pp. 33–38, 2008.
- [30] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [31] S. Hong and H. Kim, "An analytical model for a GPU architecture with memory-level and thread-level parallelism awareness," in *Proceedings of the 36th Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2009, pp. 152–163.
- [32] J.-C. Huang, J. H. Lee, H. Kim, and H.-H. S. Lee, "GPUMech: GPU performance modeling technique based on interval analysis," in *Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2014, pp. 268–279.
- [33] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," 2024. [Online]. Available: <https://arxiv.org/abs/2310.01798>
- [34] A. Ivanov, N. Dryden, T. Ben-Nun, S. Li, and T. Hoefler, "Data movement is all you need: A case study on optimizing transformers," in *Proceedings of the International Conference on Machine Learning (MLSys)*, 2021, pp. 711–722.
- [35] R. Jain, *The Art of Computer Systems Performance Analysis*. John Wiley & Sons, 1991.
- [36] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," in *ACM Computing Surveys*, vol. 55, no. 12. ACM, 2023, pp. 1–38.
- [37] N. Kang, G. Yuan, Z. Yan, B. Zhang, B. Li, Z. Li, S. Wang, G. Chen, J. Rao, Z. Wang, W. Jia, N. Sun, and G. Tan, "Md-pipe: A strong scaling enhanced pipeline architecture for ab initio accuracy molecular dynamics," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA '25. New York,

- NY, USA: Association for Computing Machinery, 2025, p. 1956–1968. [Online]. Available: <https://doi.org/10.1145/3695053.3731052>
- [38] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, “Scaling laws for neural language models,” *arXiv preprint arXiv:2001.08361*, 2020.
- [39] T. S. Karkhanis and J. E. Smith, “A first-order superscalar processor model,” in *Proceedings of the 31st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2004, pp. 338–349.
- [40] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, “Accel-sim: an extensible simulation framework for validated gpu modeling,” in *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture*, ser. ISCA ’20. IEEE Press, 2020, p. 473–486. [Online]. Available: <https://doi.org/10.1109/ISCA45697.2020.00047>
- [41] M. Kiani and A. Rajabzadeh, “Efficient cache performance modeling in GPUs using reuse distance analysis,” *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 15, no. 4, pp. 1–24, 2018.
- [42] H. Le, H. Chen, A. Saha, A. Gokul, D. Sahoo, and S. Joty, “Codechain: Towards modular code generation through chain of self-revisions with representative sub-modules,” 2024. [Online]. Available: <https://arxiv.org/abs/2310.08992>
- [43] J. Lee, Y. Ha, S. Lee, J. Woo, J. Lee, H. Jang, and Y. Kim, “GCoM: A detailed GPU core model for accurate analytical modeling of modern GPUs,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2022, pp. 424–436.
- [44] J. Lee, K. Choi, J. Lee, S. Lee, J. Whangbo, and J. Sim, “Neurex: A case for neural rendering acceleration,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA ’23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3579371.3589056>
- [45] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, “CAMEL: Communicative agents for “mind” exploration of large language model society,” in *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
- [46] S. Li, J. H. Ahn, R. D. Strong, J. B. Brockman, D. M. Tullsen, and N. P. Jouppi, “McPAT: An integrated power, area, and timing modeling framework for multicore and manycore architectures,” in *Proceedings of the 42nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2009, pp. 469–480.
- [47] W. Liang, Y. Iber, Y. Li, Z. Cao, L. Chen, T. Hashimoto, and J. Zou, “Can large language models provide useful feedback on research papers? a large-scale empirical analysis,” in *arXiv preprint arXiv:2310.01783*, 2024.
- [48] W. Liang, Y. Zhang, Z. Cao, H. Xu, Y. Yu, D. Schwieterman, J. Z. Kolter, T. Goldstein, and T. Hashimoto, “Mapping the increasing use of LLMs in scientific papers,” *arXiv preprint arXiv:2404.01268*, 2024.
- [49] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, “Let’s verify step by step,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [50] T. Liu, X. Song, Z. Yue, R. Wen, X. Hu, Z. Song, Y. Wen, Y. Hao, W. Li, Z. Du, R. Zhang, J. Guo, D. Huang, S. Peng, G. Sun, Q. Guo, and T. Chen, “Cambricon-sr: An accelerator for neural scene representation with sparse encoding table,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1254–1268. [Online]. Available: <https://doi.org/10.1145/3695053.3731018>
- [51] K. Lo, L. L. Wang, M. Neumann, R. Kinney, and D. S. Weld, “S2ORC: The semantic scholar open research corpus,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 4969–4983.
- [52] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha, “The AI scientist: Towards fully automated open-ended scientific discovery,” in *arXiv preprint arXiv:2408.06292*, 2024.
- [53] H. Luo, A. Olgun, A. G. Yaglikci, Y. Kim, and O. Mutlu, “Ramulator 2.0: A modern, modular, and extensible DRAM simulator,” in *IEEE Computer Architecture Letters*, vol. 22, no. 2, 2023, pp. 101–104.
- [54] W. Ma, Q. Xiong, X. Shi, X. Ma, H. Jin, H. Kuang, M. Gao, Y. Zhang, H. Shen, and W. Hu, “Gzkr: A gpu accelerated zero-knowledge proof system,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 340–353. [Online]. Available: <https://doi.org/10.1145/3575693.3575711>
- [55] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhume, Y. Yang *et al.*, “Self-refine: Iterative refinement with self-feedback,” in *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
- [56] D. Narayanan, M. Shoenybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kasber, H. Andriasyan, and B. Catanzaro, “Efficient large-scale language model training on GPU clusters using Megatron-LM,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021, pp. 1–15.
- [57] C. Nugteren, G.-J. van den Braak, H. Corporaal, and H. Bal, “A detailed GPU cache model based on reuse distance theory,” in *Proceedings of the 20th IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2014, pp. 37–48.
- [58] G. Offenbeck, R. Steinmann, V. Caparros, D. G. Spampinato, and M. Püschel, “Applying the roofline model,” in *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2014, pp. 76–85.
- [59] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, “Is self-repair a silver bullet for code generation?” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [60] S. Polu, J. M. Han, K. Zheng, M. Bousquet-Mélou, G. Lample, and C. Szegedy, “Formal mathematics statement curriculum learning,” *arXiv preprint arXiv:2202.01344*, 2022.
- [61] P. Ranganathan, D. Stodolsky, J. Calow, J. Dorfman, M. Guevara, C. W. Smullen IV, A. Kuusela, R. Balasubramanian, S. Bhatia, P. Chauhan, A. Cheung, I. S. Chong, N. Dasharathi, J. Feng, B. Fosco, S. Foss, B. Gelb, S. J. Gwin, Y. Hase, D.-k. He, C. R. Ho, R. W. Huffman Jr., E. Indupalli, I. Jayaram, P. Kongetira, C. M. Kyaw, A. Laursen, Y. Li, F. Lou, K. A. Lucke, J. Maaninen, R. Macias, M. Mahony, D. A. Munday, S. Muroor, N. Penukonda, E. Perkins-Argueta, D. Persaud, A. Ramirez, V.-M. Rautio, Y. Ripley, A. Salek, S. Sekar, S. N. Sokolov, R. Springer, D. Stark, M. Tan, M. S. Wachsler, A. C. Walton, D. A. Wickeraad, A. Wijaya, and H. K. Wu, “Warehouse-scale video acceleration: co-design and deployment in the wild,” in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 600–615. [Online]. Available: <https://doi.org/10.1145/3445814.3446723>
- [62] A. F. Rodrigues, K. S. Hemmert, B. W. Barrett, C. Kersey, R. Oldfield, M. Weston, R. Risen, J. Cook, P. Rosenfeld, E. Cooper-Balis *et al.*, “The structural simulation toolkit,” *ACM SIGMETRICS Performance Evaluation Review*, vol. 38, no. 4, pp. 37–42, 2011.
- [63] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. Praney Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi *et al.*, “Mathematical discoveries from program search with large language models,” in *Nature*, vol. 625, 2024, pp. 468–475.
- [64] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez *et al.*, “Code Llama: Open foundation models for code,” in *arXiv preprint arXiv:2308.12950*, 2023.
- [65] N. Samardzic, A. Feldmann, A. Krastev, S. Devadas, R. Dreslinski, C. Peikert, and D. Sanchez, “F1: A fast and programmable accelerator for fully homomorphic encryption,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 238–252. [Online]. Available: <https://doi.org/10.1145/3466752.3480070>
- [66] N. Samardzic, A. Feldmann, A. Krastev, N. Manohar, N. Genise, S. Devadas, K. Eldefrawy, C. Peikert, and D. Sanchez, “Craterlake: a hardware accelerator for efficient unbounded computation on encrypted data,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ser. ISCA ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 173–187. [Online]. Available: <https://doi.org/10.1145/3470496.3527393>
- [67] D. Sanchez and C. Kozyrakis, “ZSim: Fast and accurate microarchitectural simulation of thousand-core systems,” in *Proceedings of the 40th Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2013, pp. 475–486.
- [68] K. Sankaralingam, “Computer architecture’s alphazero moment: Automated discovery in an encircled world,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.03312>
- [69] D. E. Shaw, M. M. Deneroff, R. O. Dror, J. S. Kuskin, R. H. Larson, J. K. Salmon, C. Young, B. Batson, K. J. Bowers, J. C. Chao, M. P.

- Eastwood, J. Gagliardo, J. P. Grossman, C. R. Ho, D. J. Ierardi, I. Kolossváry, J. L. Klepeis, T. Layman, C. McLeavey, M. A. Moraes, R. Mueller, E. C. Priest, Y. Shan, J. Spengler, M. Theobald, B. Towles, and S. C. Wang, “Anton, a special-purpose machine for molecular dynamics simulation,” *Commun. ACM*, vol. 51, no. 7, p. 91–97, Jul. 2008. [Online]. Available: <https://doi.org/10.1145/1364782.1364802>
- [70] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
- [71] J. Sim, A. Dasgupta, H. Kim, and R. Vuduc, “A performance analysis framework for identifying potential benefits in GPGPU applications,” in *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*. ACM, 2012, pp. 11–22.
- [72] R. Taylor, M. Kardas, G. Cucurull, T. Scialom, A. Hartshorn, E. Saravia, A. Poulton, V. Kerkez, and R. Stojnic, “Galactica: A large language model for science,” *arXiv preprint arXiv:2211.09085*, 2022.
- [73] T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong, “Solving olympiad geometry without human demonstrations,” in *Nature*, vol. 625, 2024, pp. 476–482.
- [74] Y. Turakhia, G. Bejerano, and W. J. Dally, “Darwin: A genomics co-processor provides up to 15,000x acceleration on long read assembly,” in *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS ’18. New York, NY, USA: Association for Computing Machinery, 2018, p. 199–213. [Online]. Available: <https://doi.org/10.1145/3173162.3173193>
- [75] O. Villa, D. Lustig, Z. Yan, E. Bolotin, Y. Fu, N. Chatterjee, N. Jiang, and D. Nellans, “Need for speed: Experiences building a trustworthy system-level GPU simulator,” in *Proceedings of the 27th IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 868–880.
- [76] D. Wadden, S. Lin, K. Lo, L. L. Wang, M. van Zuylen, A. Cohan, and H. Hajishirzi, “Fact or fiction: Verifying scientific claims,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 7534–7550.
- [77] L. Wang, M. Jahre, A. Adileh, and L. Eeckhout, “MDM: The GPU memory divergence model,” in *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 1089–1101.
- [78] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, “Executable code actions elicit better LLM agents,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- [79] Y. Wang, Q. Guo, W. Yao, H. Zhang, X. Zhang, Z. Wu, M. Zhang, X. Dai, M. Zhang, Q. Wen, W. Ye, S. Zhang, and Y. Zhang, “Autosurvey: Large language models can automatically write surveys,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.10252>
- [80] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Proceedings of the 36th Conference on Neural Information Processing Systems (NeurIPS)*, 2022.
- [81] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” in *Communications of the ACM*, vol. 52, no. 4. ACM, 2009, pp. 65–76.
- [82] D. Wright and I. Augenstein, “Citeworth: Cite-worthiness detection for improved scientific document understanding,” in *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, 2021, pp. 1796–1807.
- [83] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu *et al.*, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” in *arXiv preprint arXiv:2308.08155*, 2023.
- [84] K. Yang, A. M. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. Prenger, and A. Anandkumar, “LeanDojo: Theorem proving with retrieval-augmented language models,” *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
- [85] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” in *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS)*, 2024.
- [86] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
- [87] Y. Zhang and J. D. Owens, “A quantitative performance analysis model for GPU architectures,” in *Proceedings of the 17th IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2011, pp. 382–393.
- [88] Y. Zhang, S. Wang, X. Zhang, J. Dong, X. Mao, F. Long, C. Wang, D. Zhou, M. Gao, and G. Sun, “Pipezk: accelerating zero-knowledge proof with a pipelined architecture,” in *Proceedings of the 48th Annual International Symposium on Computer Architecture*, ser. ISCA ’21. IEEE Press, 2021, p. 416–428. [Online]. Available: <https://doi.org/10.1109/ISCA52012.2021.00040>
- [89] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing *et al.*, “Judging LLM-as-a-judge with MT-Bench and chatbot arena,” in *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
- [90] D. Zhu, R. Meng, Y. Song, X. Wei, S. Li, T. Pfister, and J. Yoon, “Paperbanana: Automating academic illustration for ai scientists,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.23265>